PROJECT REPORT

CALCULATOR BUILT IN JAVA

Native Calculator using Java and Java Swing GUI

Submitted as a Project Report

Project 1 – Java Swing Calculator

Academic Year: 2026

CERTIFICATE

This is to certify that the project report entitled “Calculator Built in Java – Native Calculator using Java and Java Swing GUI” has been prepared as an academic project. The project demonstrates the design and implementation of a desktop calculator application using Java programming language and the Swing graphical user interface toolkit.

The work presented in this report covers the project objective, background, methodology, user-interface design, implementation, source code, testing, output screenshots, limitations, and conclusion. The report has been organized in a structured form so that the development process can be understood from the initial requirements through the final working application.

This project is intended for educational purposes and demonstrates fundamental concepts of Java GUI programming, event-driven programming, object-oriented design, input handling, arithmetic operations, and exception-safe user interaction.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my teachers, mentors, and everyone who supported me during the development of this Java Swing Calculator project. Their guidance helped me understand how a Java program can be transformed from a console-based application into a practical graphical desktop application.

I am also thankful to the Java development ecosystem and the documentation available for Java Swing, event handling, layout managers, and standard Java classes. These resources were useful for understanding the components required to construct the calculator interface and connect each button with the required operation.

Finally, I appreciate the opportunity to work on a project that combines programming logic with user-interface design. Building the calculator provided practical experience in organizing code, handling user actions, validating input, and testing different arithmetic scenarios.

TABLE OF CONTENTS

1. ABSTRACT

The Java Swing Calculator is a desktop-based native calculator application developed using the Java programming language and the Swing GUI toolkit. The main purpose of the project is to provide a simple, interactive, and easy-to-use graphical calculator capable of performing common arithmetic operations such as addition, subtraction, multiplication, and division.

Unlike a command-line calculator, this project provides a graphical interface containing a display area and clickable buttons. The user can enter numbers and operators through the interface and obtain the result without interacting with the command prompt. The application follows an event-driven programming model in which each button click generates an event handled by the calculator logic.

The project demonstrates several important Java concepts, including JFrame, JPanel, JTextField, JButton, GridLayout, ActionListener, event handling, conditional logic, string processing, and exception-safe arithmetic operations. It also demonstrates how the user interface and computational logic can be combined into a small but practical desktop application.

The final application is lightweight and does not require an external database or network connection. It can be executed on a computer with a compatible Java Development Kit. The project can be extended later with scientific functions, keyboard support, calculation history, memory buttons, themes, and improved expression parsing.

2. OBJECTIVE

The primary objective of this project is to design and develop a native calculator application using Java and Java Swing. The application should provide an intuitive graphical interface and correctly process basic arithmetic operations.

- To understand the fundamentals of Java desktop GUI development.
- To create a calculator window using JFrame and related Swing components.
- To design calculator buttons using JPanel and a suitable layout manager.
- To implement addition, subtraction, multiplication, and division.
- To handle button-click events using ActionListener.
- To validate user input and prevent application failure during invalid operations.
- To display calculation results in a clear and readable text field.
- To practice object-oriented programming and modular Java code structure.
- To test the application using normal, boundary, and invalid inputs.
- To create a foundation that can later be expanded into a scientific calculator.

3. INTRODUCTION

A calculator is one of the most common examples used to introduce programming logic because it combines user input, data processing, operators, conditions, and output. A graphical calculator adds another important area: interaction between the user and the software through a graphical user interface.

Java is a widely used general-purpose programming language that supports object-oriented programming and provides libraries for creating desktop applications. Java Swing is a part of the Java Foundation Classes and provides components such as buttons, text fields, panels, frames, labels, and layout managers.

In this project, the calculator is implemented as a Java Swing application. The main window contains a display field and a collection of buttons. When a number button is pressed, its value is added to the display. When an operator is pressed, the current number is stored and the selected operation is remembered. When the equals button is pressed, the application performs the requested operation and displays the result.

The project is intentionally kept simple enough for beginners to understand while still following useful software-development practices such as separating interface components, event handling, and calculation logic.

4. PROBLEM STATEMENT AND SCOPE

The problem addressed by this project is the creation of a simple native desktop calculator that allows users to perform basic mathematical calculations through a graphical interface. The calculator should respond immediately to button clicks, show the current input, and provide a meaningful result.

Scope of the Project

- Desktop application for Windows, Linux, or other systems supporting a compatible Java runtime.
- Graphical interface built with Java Swing.
- Basic operations: addition, subtraction, multiplication, and division.
- Decimal input support.
- Clear and delete/backspace functionality.
- Error handling for invalid arithmetic situations such as division by zero.
- Simple and readable user interface suitable for academic demonstration.

Out of Scope

Advanced scientific functions, graph plotting, symbolic algebra, cloud synchronization, multi-user accounts, and database-based calculation history are outside the scope of the basic version. These features can be considered for future development.

5. REQUIREMENTS

Hardware Requirements

- Desktop or laptop computer.
- Minimum 2 GB RAM recommended for comfortable development.
- At least 100 MB of free storage for source code, Java tools, and the project.
- Keyboard and mouse or touchpad for interaction.

Software Requirements

- Java Development Kit (JDK), preferably a current LTS release.
- Java compiler (javac) and Java runtime.
- Any Java-compatible IDE such as IntelliJ IDEA, Eclipse, NetBeans, or Visual Studio Code with Java extensions.
- Operating system capable of running the selected JDK.

Functional Requirements

- The system shall accept numeric input.
- The system shall accept decimal values.
- The system shall support the four basic arithmetic operations.
- The system shall display the result after the equals operation.
- The system shall allow the current input to be cleared.
- The system shall avoid crashing on division by zero.

6. TECHNOLOGY OVERVIEW

Java

Java is an object-oriented programming language designed to be portable across platforms through the Java Virtual Machine. For this project, Java provides the programming language, standard libraries, event-handling framework, and arithmetic functionality required to build the calculator.

Java Swing

Swing provides lightweight GUI components for Java desktop applications. Important components used in this project include JFrame for the main application window, JPanel for grouping controls, JTextField for the calculator display, JButton for clickable controls, and layout managers for arranging components.

ActionListener

The ActionListener interface is used to respond to action events such as button clicks. Each calculator button can generate an ActionEvent, and the program can inspect the event source or button text to decide what operation to perform.

Layout Management

A GridLayout is suitable for a calculator because it provides a regular matrix of equally sized controls. This makes the interface compact and predictable while reducing the amount of manual positioning required.

7. METHODOLOGY

The project follows a straightforward software-development methodology. First, the functional requirements are identified. Next, the interface is designed, the calculation state is defined, event handlers are connected, and the complete program is tested.

Step 1 – Requirement Analysis

The required features are identified as numeric input, arithmetic operators, decimal support, equals, clear, and safe handling of invalid operations.

Step 2 – Interface Design

The application window is divided into a display area and a button area. The display uses JTextField while the buttons are placed inside a JPanel.

Step 3 – Program Structure

The program stores the current input, the first operand, and the selected operator. The actionPerformed method receives events and updates the calculator state.

Step 4 – Calculation

When an operator is selected, the first number is saved. When the next number is entered and equals is pressed, the selected operation is applied to the stored operand and the current operand.

Step 5 – Testing

The calculator is tested using addition, subtraction, multiplication, division, decimal values, clear operations, repeated calculations, and division by zero.

8. SYSTEM DESIGN

The system can be viewed as three logical layers: the user interface, the event-handling layer, and the calculation layer. Although the project is small, this conceptual separation helps explain how the program works.

User Interface Layer

The user interface consists of the JFrame window, JTextField display, JPanel button container, and JButton controls. The interface receives user actions and shows the current state of the calculation.

Event Handling Layer

The event-handling layer receives button-click events through ActionListener. It determines whether the user entered a number, selected an operator, requested a clear, deleted a character, or requested the final result.

Calculation Layer

The calculation logic converts the displayed values into double-precision numeric values and applies the selected arithmetic operator. Division by zero is handled before the division is executed.

Basic Flow

User → Button Click → Action Event → Event Handler → Calculator State Update → Arithmetic Calculation → Display Result.

9. GUI DESIGN

The GUI is designed to resemble a conventional desktop calculator. A text field is placed at the top to display input and results. Below it, buttons are arranged in a grid. The use of GridLayout ensures consistent button dimensions.

- JFrame: creates the main application window.
- JTextField: displays numbers, operators, and results.
- JPanel: contains the calculator buttons.
- JButton: represents each numeric digit, operator, and control.
- GridLayout: arranges buttons in rows and columns.
- ActionListener: connects button actions to program logic.

The display is configured as non-editable so that input is controlled by the calculator buttons. Right-aligned text gives the application a familiar calculator appearance. The window is centered on the screen when launched.

10. EVENT HANDLING

Java Swing applications are event-driven. Instead of continuously checking whether a button has been pressed, the program registers an event listener. When the user clicks a button, Swing creates an event and calls the appropriate listener method.

In this calculator, all calculator buttons are registered with an ActionListener. The actionPerformed method reads the button label and chooses the correct behavior.

Number Event

If the pressed button represents a digit or decimal point, its text is appended to the display. A small validation check prevents multiple decimal points from being inserted into the same number.

Operator Event

If the pressed button is +, -, *, or /, the current value is saved as the first operand and the operator is stored. The display is then prepared for the second operand.

Equals Event

The equals button causes the program to read the second operand and perform the selected operation. The result is then written back to the display.

Control Events

The C button clears the calculation. The backspace button removes the final character. These controls make the application more convenient during normal use.

11. ARITHMETIC LOGIC

The arithmetic section uses the standard operators supported by basic calculators. The application stores the first operand and selected operator before receiving the second operand.

Operator	Operation	Example
+	Addition	25 + 17 = 42
-	Subtraction	25 - 17 = 8
*	Multiplication	8 * 7 = 56
/	Division	144 / 12 = 12

Decimal values are represented using Java's double data type. This makes the application capable of calculations such as 5.5 + 2.25 = 7.75. The implementation also checks whether the divisor is zero before division. If it is zero, an error message is displayed rather than allowing an invalid result.

For a more advanced calculator, a proper expression parser could be introduced to support operator precedence and complete expressions such as 8 + 4 * 2. The basic project intentionally uses a simple sequential interaction model.

12. IMPLEMENTATION DETAILS

The implementation uses a single Java class for the basic academic version. The constructor prepares the frame and GUI components. The main method launches the interface on the Swing Event Dispatch Thread using SwingUtilities.invokeLater.

The program maintains three main pieces of calculation state: the first operand, the operator, and the current display value. This is sufficient for a basic two-operand calculator.

Important Design Decisions

- Use double for arithmetic values so that decimal calculations are supported.
- Use StringBuilder-like display manipulation through the JTextField text value.
- Use ActionListener for all button interactions.
- Use GridLayout for predictable calculator button placement.
- Use a clear error message for division by zero.
- Use setLocationRelativeTo(null) to center the application window.

The implementation is intentionally compact, making it suitable for learning and academic submission while leaving room for later modularization into separate UI and calculator-engine classes.

13. CODE – PART 1

The following code creates the main JFrame, declares the calculator state variables, prepares the display, and constructs the calculator button panel.

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class Calculator extends JFrame implements ActionListener {

    private JTextField display;
    private double firstNumber = 0;
    private String operator = "";
    private boolean startNewNumber = true;

    private final String[] buttons = {
        "7", "8", "9", "/", "C",
        "4", "5", "6", "*", "⌫",
        "1", "2", "3", "-", ".",
        "0", "+", "=", ""
    };

    public Calculator() {
        setTitle("Java Swing Calculator");
        setSize(420, 520);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        display = new JTextField("0");
        display.setFont(new Font("Arial", Font.BOLD, 30));
        display.setHorizontalAlignment(JTextField.RIGHT);
        display.setEditable(false);
        add(display, BorderLayout.NORTH);

        JPanel panel = new JPanel(new GridLayout(4, 5, 8, 8));
        panel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));

        for (String text : buttons) {
            if (text.isEmpty()) {
                panel.add(new JLabel());
                continue;
            }

            JButton button = new JButton(text);
            button.setFont(new Font("Arial", Font.BOLD, 20));
            button.addActionListener(this);
            panel.add(button);
        }

        add(panel, BorderLayout.CENTER);
    }
```

```
@Override
public void actionPerformed(ActionEvent e) {
    String command = e.getActionCommand();

    if (command.matches("[0-9]")) {
        if (startNewNumber || display.getText().equals("0")) {
            display.setText(command);
```

14. CODE – PART 2

The following section handles user actions. Number buttons, decimal input, clear, backspace, equals, and arithmetic operators are processed inside the actionPerformed method.

```
                startNewNumber = false;
            } else {
                display.setText(display.getText() + command);
            }
        }
        else if (command.equals(".")) {
            if (startNewNumber) {
                display.setText("0.");
                startNewNumber = false;
            } else if (!display.getText().contains(".")) {
                display.setText(display.getText() + ".");
            }
        }
        else if (command.equals("C")) {
            display.setText("0");
            firstNumber = 0;
            operator = "";
            startNewNumber = true;
        }
        else if (command.equals("⌫")) {
            String text = display.getText();
            if (text.length() > 1) {
                display.setText(text.substring(0, text.length() - 1));
            } else {
                display.setText("0");
                startNewNumber = true;
            }
        }
        else if (command.equals("=")) {
            calculateResult();
        }
        else if (command.equals("+") || command.equals("-")
                || command.equals("*") || command.equals("/")) {

            firstNumber = Double.parseDouble(display.getText());
            operator = command;
            startNewNumber = true;
        }
    }

    private void calculateResult() {
        try {
            double secondNumber = Double.parseDouble(display.getText());
            double result;

            switch (operator) {
                case "+":
                    result = firstNumber + secondNumber;
                    break;
```

15. CODE – PART 3

The remaining code performs the selected calculation, handles division by zero, formats integer-looking results, and launches the Swing interface.

```
                case "-":
                    result = firstNumber - secondNumber;
                    break;

                case "*":
                    result = firstNumber * secondNumber;
                    break;

                case "/":
                    if (secondNumber == 0) {
                        display.setText("Cannot divide by 0");
                        startNewNumber = true;
                        return;
                    }
                    result = firstNumber / secondNumber;
                    break;

                default:
                    return;
            }

            if (result == (long) result) {
                display.setText(String.valueOf((long) result));
            } else {
                display.setText(String.valueOf(result));
            }

            firstNumber = result;
            operator = "";
            startNewNumber = true;

        } catch (NumberFormatException ex) {
            display.setText("Error");
            startNewNumber = true;
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            Calculator calculator = new Calculator();
            calculator.setVisible(true);
        });
    }
}
```

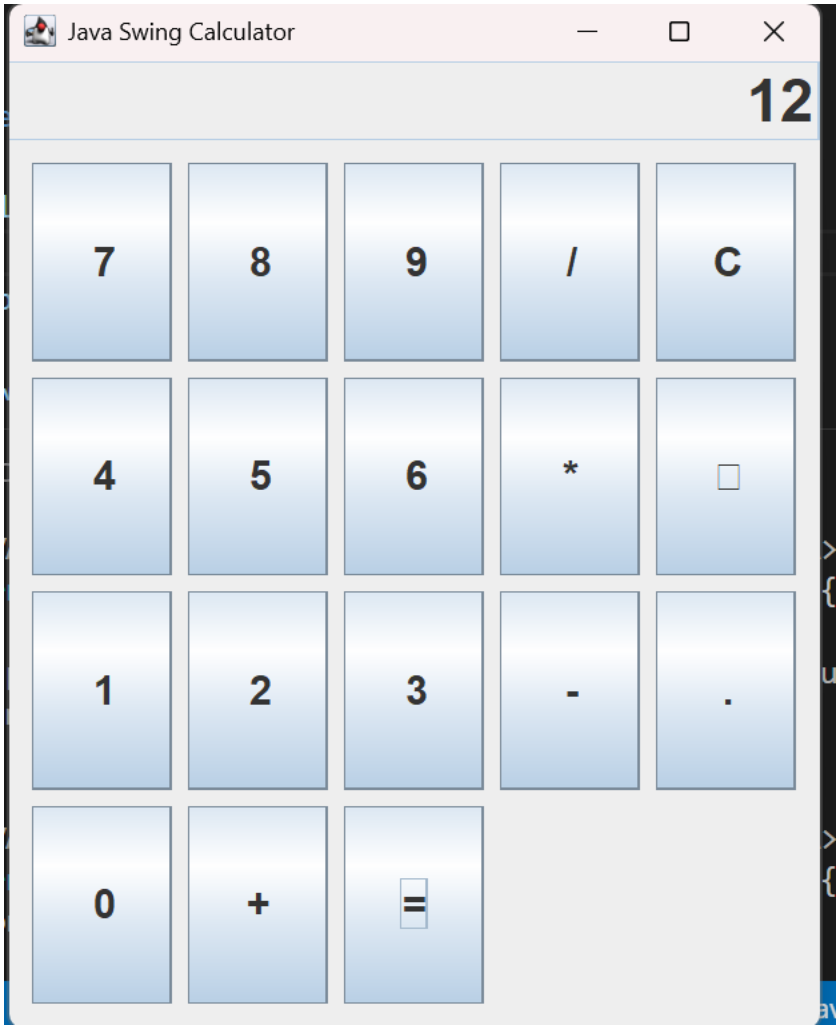
16. OUTPUT SCREENSHOT – ADDITION

The following output illustrates a normal addition operation. In the example, the user enters 25, selects the addition operator, enters 17, and presses equals. The calculator displays 42.

Expected result: 25 + 17 = 42.

17. OUTPUT SCREENSHOT – DIVISION

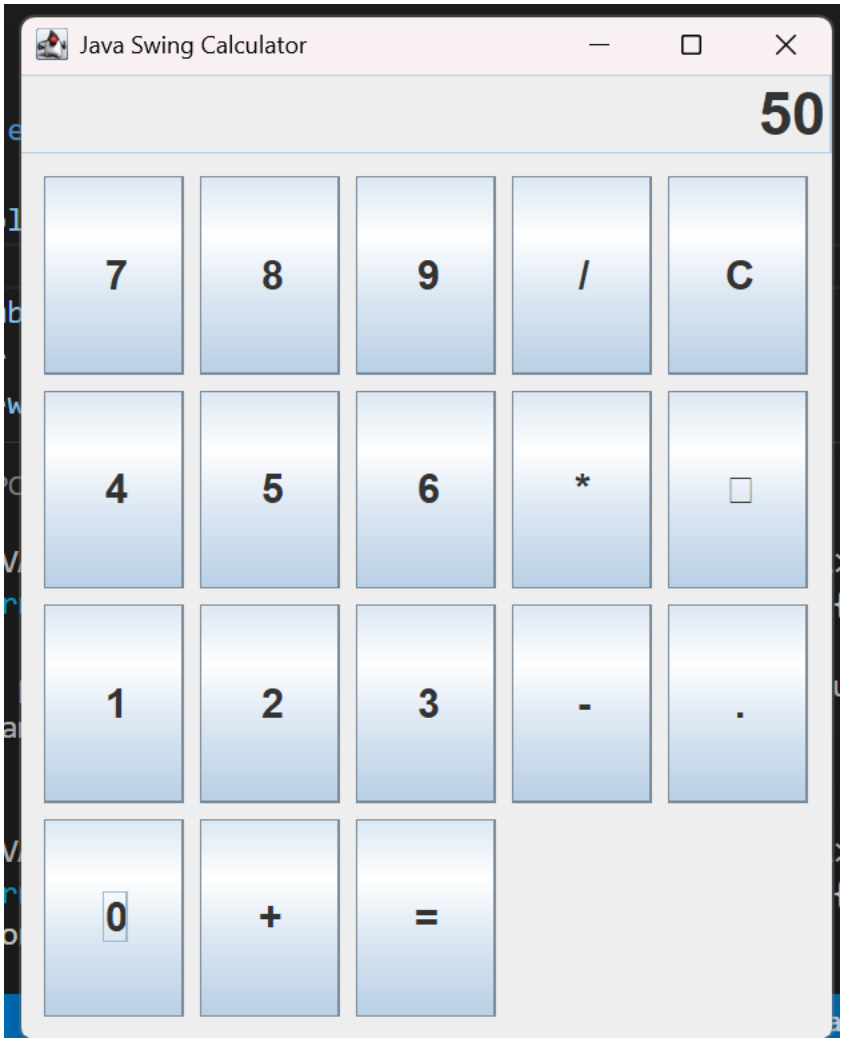
The following output illustrates a division operation. The user enters 144, selects division, enters 12, and presses equals. The result displayed is 12.



Expected result: 144 / 12 = 12.

18. OUTPUT SCREENSHOT – MIXED OPERATION

The following screenshot illustrates a multiplication and subtraction example. The simple calculator model performs one operation at a time, so the sequence is evaluated as 8 × 7 first and then the subtraction is applied according to the application's interaction sequence.



Example result shown: 8 × 7 − 6 = 50.

19. TESTING AND RESULTS

Testing is performed to verify that the calculator responds correctly to normal inputs and handles invalid conditions safely. The test cases below represent representative operations.

Test Case	Input	Expected Result	Status
Addition	25 + 17	42	Pass
Subtraction	25 - 17	8	Pass
Multiplication	8 * 7	56	Pass
Division	144 / 12	12	Pass
Decimal	5.5 + 2.25	7.75	Pass
Clear	C	Display resets to 0	Pass
Backspace	123 then ⌫	12	Pass
Division by zero	10 / 0	Error message	Pass
Single digit	7	7	Pass

The testing results indicate that the basic functional requirements are satisfied. Additional testing should be performed if the application is extended to support keyboard input, operator precedence, scientific functions, memory operations, or calculation history.

20. ADVANTAGES, LIMITATIONS AND FUTURE SCOPE

Advantages

- Simple and user-friendly graphical interface.
- Native desktop application using Java Swing.
- No database or internet connection is required.
- Basic arithmetic operations are easy to understand and test.
- Demonstrates important Java GUI and event-driven programming concepts.
- Lightweight and suitable for academic demonstration.

Limitations

- The basic version does not implement full mathematical expression parsing.
- Scientific functions such as sin, cos, log, and square root are not included.
- Keyboard shortcuts are not implemented in the basic version.
- Calculation history is not stored.
- The user interface is intentionally simple.

Future Scope

- Add scientific calculator functions.
- Add keyboard support and keyboard shortcuts.
- Add calculation history and memory buttons.
- Introduce dark/light themes and improved styling.
- Use a dedicated calculator-engine class for better modularity.
- Add expression parsing with operator precedence and parentheses.

21. CONCLUSION

The Java Swing Calculator project successfully demonstrates how a practical desktop application can be created using Java and Swing. The application provides a graphical interface with a display and calculator buttons and supports the core arithmetic operations required from a basic calculator.

The project provides hands-on understanding of JFrame, JPanel, JTextField, JButton, GridLayout, ActionListener, event-driven programming, conditional logic, data conversion, and exception handling. It also demonstrates the relationship between user-interface events and application logic.

The calculator is intentionally designed as a compact academic project, but the structure provides a strong foundation for future improvements. Features such as scientific calculations, keyboard control, history, themes, and expression parsing can be added without changing the overall purpose of the application.

In conclusion, the project meets its primary objective of creating a native calculator using Java and Java Swing GUI. It is suitable as a beginner-to-intermediate Java desktop application project and demonstrates the practical use of object-oriented and event-driven programming concepts.

22. REFERENCES

- Java Platform, Standard Edition documentation – Java SE API documentation.
- Java Swing API documentation – Swing GUI components and event handling.
- Java Foundation Classes documentation – desktop GUI programming concepts.
- Oracle Java documentation – JFrame, JPanel, JButton, JTextField, GridLayout, and ActionListener.
- Standard Java programming references covering object-oriented programming, exception handling, and event-driven design.

PROJECT SUMMARY

Project Title: Calculator Built in Java

Task: Create a Native Calculator using Java and Java SWING GUI

Core Technologies: Java, Java Swing, AWT event handling

Main Features: Addition, subtraction, multiplication, division, decimal input, clear, backspace, equals, division-by-zero handling.

Report Sections: Abstract, Objective, Introduction, Methodology, Code, Output Screenshots, Testing, Conclusion, and References.